



Actividad de Clase: Diseño y consulta de una base de datos de ventas en línea

Caso práctico para identificar elementos clave de una base de datos relacional y ejecutar consultas básicas en SQL para obtener información de valor empresarial.

Objetivo y Preparación

Identificar elementos clave

Tablas, llaves, relaciones y vistas de una base de datos relacional

Diseñar modelo de datos

Crear un modelo simplificado a partir de un caso realista de comercio electrónico

Ejecutar consultas SQL

Realizar consultas básicas para obtener información de valor empresarial

Materiales y preparación previa

-  Haber leído el documento de apoyo: Elementos de una base de datos y SQL
-  Contar con un entorno de práctica SQL (PostgreSQL, MySQL, SQLite o simulador en línea como SQL Fiddle)
-  Conexión a internet y editor de texto/notebook para registrar respuestas

Requisitos técnicos

Entorno SQL básico (pgAdmin, DBeaver, SQLite Studio o un sandbox online).

Copia del enunciado y espacio para escribir código SQL.

Enunciado del Reto y Desarrollo

Una empresa de comercio electrónico necesita organizar su información de clientes, productos y pedidos. Actualmente guardan datos en hojas sueltas, lo que dificulta los reportes de ventas.

Tu misión como analista es:

- Definir un modelo de base de datos relacional mínimo
- Crear las tablas en SQL con sus llaves
- Insertar datos de prueba
- Ejecutar consultas para analizar las ventas por cliente

Instrucciones paso a paso

01

Diseño conceptual (5')

Identifica las entidades necesarias: Clientes, Productos, Pedidos.

Relaciona: un cliente puede hacer muchos pedidos (1:N) y un pedido contiene varios productos (N:M → tabla intermedia).

Micro-check : dibuja un mini DER en tu cuaderno (Cliente–Pedido–Producto).

02

Creación de tablas en SQL (10')

Ejecuta los siguientes comandos:

```
CREATE TABLE clientes (  
  id_cliente SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  correo VARCHAR(100) UNIQUE,  
  fecha_registro DATE DEFAULT CURRENT_DATE  
);
```

```
CREATE TABLE productos (  
  id_producto SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  precio NUMERIC(10,2) CHECK (precio > 0)  
);
```

```
CREATE TABLE pedidos (  
  id_pedido SERIAL PRIMARY KEY,  
  id_cliente INT REFERENCES clientes(id_cliente),  
  fecha DATE DEFAULT CURRENT_DATE,  
  monto NUMERIC(10,2)  
);
```

Micro-check : verifica que las tablas se crean sin errores.

03

Insertión de registros (5')

```
INSERT INTO clientes (nombre, correo) VALUES  
( 'Ana Torres', 'ana@mail.com'),  
( 'Luis Pérez', 'luis@mail.com');
```

```
INSERT INTO productos (nombre, precio) VALUES  
( 'Teclado', 20000),  
( 'Mouse', 10000),  
( 'Monitor', 120000);
```

```
INSERT INTO pedidos (id_cliente, fecha, monto)  
VALUES  
(1, '2024-09-01', 140000),  
(2, '2024-09-02', 10000);
```

Micro-check : confirma con `SELECT * FROM clientes;` que los registros están insertados.

04

Consulta analítica (10')

Ejecuta:

```
SELECT c.nombre, SUM(p.monto) AS total_compras  
FROM clientes c  
JOIN pedidos p ON c.id_cliente = p.id_cliente  
GROUP BY c.nombre;
```

✓ Este reporte muestra cuánto ha comprado cada cliente.

Micro-check : interpreta resultados: ¿quién gastó más?

Criterios de Éxito y Entrega

Criterios de éxito

- Tablas creadas con llaves primarias y foráneas
- Inserción de datos válida (sin errores de integridad)
- Consultas ejecutadas correctamente y resultados interpretados
- Uso de al menos una vista como extensión

Errores comunes

- No definir llaves primarias o relaciones
- Insertar datos con tipos incorrectos (ej: precio negativo)
- Omitir JOIN, lo que genera resultados incorrectos

Formato de entrega

Un archivo .sql o capturas de pantalla del entorno.

Una breve conclusión escrita: "El cliente que más compró fue ___ con un total de ___".

 **Tiempo total: 45 minutos**

